



Adaptive Fingerprinting: HTTP-Basma's Multi-Stage Probing for Granular Server Differentiation

Mohamad Mokbel
mfmokbel@netomize.ca
<https://www.netomize.ca>

May 20, 2026

Abstract

In the realm of cybersecurity, accurately identifying and characterizing web servers is crucial for threat detection, vulnerability assessment, and network mapping. We introduce HTTP-Basma, a novel active fingerprinting algorithm that unveils unique server profiles through a multi-layered approach to address this challenge.

Key Features:

Crafted Requests, Revealing Responses: HTTP-Basma sends 8 meticulously designed HTTP probes, eliciting distinctive responses that reflect server configurations.

Dual Hashing for Versatility. The algorithm generates two hashes:

- A 38-byte fuzzy hash, "verbosus", offering reversibility
- A 16-byte one-way hash, "pacto", derived from verbosus, enhancing privacy and security

Clustering and Hunting: These hashes empower server clustering, identification of unique and similar servers, and the pursuit of malicious actors with heightened confidence.

Modular Design for Expansion: The algorithm's architecture fosters the addition of new hashing variants, encouraging collaboration and adaptability.

In this paper, we first survey some of the existing notable work in HTTP fingerprinting and then explore the algorithm's functionality, design, architecture, and outcomes. Additionally, we will showcase some compelling findings from scanning the top 1 million Majestic websites, including the identification and clustering of various malware families' C&C HTTP servers. The source code and supporting data will be made available.

Related Work

This section reviews some of the notable work in HTTP fingerprinting.

Two notable tools for HTTP server fingerprinting are Net-Square's `httpprint` by (Shah, 2004) and CompuTech's `httprecon` by (Ruef, 2007). Saumil Shah developed `httpprint`, a closed-source tool that creates a 296-character hexadecimal signature using decision-tree, statistical, and neural network methods. While it shares some similarities with HTTP-Basma in its request crafting, its underlying logic, detection, and encoding are distinct. Marc Ruef's `httprecon` is an open-source alternative that sends nine HTTP requests, similar to HTTP-Basma, and analyzes ~23 indicators in server response headers. Both tools generate a "scoring" number to match against known server fingerprints, but neither produces a complex, modular and reversible fingerprint like HTTP-Basma.

WebServerFP, developed by (Kotarak, 2003), represents an older and less advanced approach to HTTP server fingerprinting. This tool's default operation involves sending four specialized requests to a server. It then attempts to identify the server by checking the order of HTTP headers and the returned status code against a pre-existing list of known signatures. Unfortunately, this method is notably prone to generating false positives.

HMAP, by (Lee, 2001). Lee published the theory and details behind the tool¹ in a paper titled "Detecting and Defending against Web-Server Fingerprinting" in February 2002, which is based on the author's master's thesis. The tool is more than 2 decades old, but it is very exhaustive in terms of the number of well-formed and malformed HTTP requests it generates to determine a best match against known and existing server signatures (they are called profiles by the author). It sends a little over 130 requests to the server. To come up with a matching score, it checks the server response for certain 123 characteristics that the author divides into three categories depending on the request type: lexical (expected status code and reason phrase), semantic (expected static code, server response when sending long URLs, large number of headers...) and syntactic (a predefined list of allow methods in a specific order, header order, ETag...). The output

¹ <https://github.com/Mebus/hmap/tree/master>

consists of the number of characteristics that match, mismatches and unknowns, enabling the identification of the server's name, version, and operating system.

Documentation - Internals

HTTP-Basma's algorithm's core idea centers on sending 8 specially crafted HTTP requests with different requirements to solicit different responses from the server. Once the server response is retrieved, the HTTP status line is surgically dissected for all elements and encoded optimally. Additionally, select headers from the server response are checked for encoding as well.

The terms HTTP 'verb' and 'method' are used synonymously in this document.

HTTP Requests/Probes

Setting the Request-Line.

The default setup only includes the headers specified in each probe. The corresponding HTTP server responses are documented in the "HTTP Responses" section later in this paper.

All requests are sent independently of each other, without reusing the same socket. The DNS cache is cleared between every request.

1. P1 - GET Normal - Valid Request

The purpose of this standard conformant request is to generate a ground-truth hash against which all other requests will be evaluated.

```
GET / HTTP/1.1
Host: <example.com>
```

2. P2 - GET Invalid HTTP Version Request

This request inexplicably specifies the non-standard HTTP version “**4.2**”, seemingly just to be different from a defined protocol, in an attempt to gauge the server response for any differences to the first request.

```
GET / HTTP/4.2
Host: <example.com>
```

3. P3 - GET Random Resource Request

This request attempts to access a non-existent resource on the server by sending a pseudo-randomly generated target resource using the following character class **[A-Za-z0-9]{16}** pattern.

Servers usually respond differently when attempting to request a non-existent resource.

```
GET /<wAs49YmHZ0PSFKGo> HTTP/1.1
Host: <example.com>
```

4. P4 - Random Verb Request

This request employs a pseudo-randomly generated 10-character uppercase verb (matching the pattern **[A-Z]{10}**). The purpose of sending this request with a non-standard verb is to observe how the server handles and responds to an unrecognized HTTP method.

```
<MBBVEIIEXE> / HTTP/1.1
Host: <example.com>
```

5. P5 - get Lowercase Verb

This request uses the HTTP GET verb in all lowercase ("**get**"). While technically a valid verb, many servers are configured only to accept it in uppercase ("GET"). They would likely reject this request, responding with a unique HTTP status-line and response headers different from the other probes.

```
get / HTTP/1.1
Host: <example.com>
```

6. P6 - GET Request - Accept-Encoding - Full

This request sends to the server a list of compression algorithms the sender understands, in this specific order: aes128gcm, br, **<grease_A>**, compress, deflate, exi, **<grease_B>**, gzip, pack200-gzip, x-compress, x-gzip, zstd, identity

Note the placeholders **<grease_A>** and **<grease_B>**, which are pseudo-randomly generated using the following character classes:

<grease_A>: [a-z]{8}

<grease_B>: [a-z0-9]{10}

The server should never pick either of the grease values. The rest of the compression algorithms, including the identity value/function are standardized.

```
GET / HTTP/1.1
Host: <example.com>
```

```
Accept-Encoding: aes128gcm, br, <grease_A>, compress, deflate, exi,  
<grease_B>, gzip, pack200-gzip, x-compress, x-gzip, zstd, identity
```

7. P7 - GET Request - Accept-Encoding - Less

This request sends to the server a reduced list of compression algorithms the sender understands, in this specific order: `<grease_A>`, `compress`, `deflate`, `<grease_B>`, `x-compress`, `zstd`, `br`

Since the `gzip` compression algorithm is the most likely to be picked up by the server, this request intends to force the server to behave differently compared to Request 6.

Note the placeholders `<grease_A>` and `<grease_B>`, which are pseudo-randomly generated using the following character classes:

```
<grease_A>: [a-z]{8}
```

```
<grease_B>: [a-z0-9]{10}
```

The server should never pick either of the grease values. The rest of the compression algorithms are standardized, albeit less used compared to `gzip`.

```
GET / HTTP/1.1  
Host: <example.com>  
Accept-Encoding: <grease_A>, compress, deflate, <grease_B>, x-compress, zstd,  
br
```

8. P8 - OPTIONS Request

This request with the `OPTIONS` method queries the server for the allowed HTTP methods for a given resource.

```
OPTIONS / HTTP/1.1  
Host: <example.com>
```

HTTP Responses

Following each request, the server's response is analyzed to extract specific headers and their values. This extracted data undergoes further processing, including dissection and encoding, to generate a reversible fingerprint.

In the event of a failed request, whether due to a lack of server response or any other reason, the resulting fingerprint for that probe will be a sequence of all zeros.

Status-Line Fingerprint (4-byte)

This generates a final 4-byte (8 hexadecimal characters) fingerprint.

For P1-P5 and P8 probes, the HTTP Status Line (also known as the Start-Line) attributes are parsed and encoded according to the following steps:

First, let's assume the server responded with this Status Line after sending probe P1:

```
HTTP/1.1 200 OK\r\n
```

The HTTP Status Line consists of these attributes (sequence of octets), each separated by a space, and the line ending with CRLF.

```
<HTTP-Version><SP><Status-Code><SP><Reason-Phrase>\r\n
```

The Reason message is optional, with the grammar `*( HTAB / SP / VCHAR / obs-text )`.

HTTP-Version Fingerprint (1-byte)

The HTTP-Version is encoded per the following pseudo-code:

```
1. // If the HTTP-Version doesn't exist, or, if the HTTP-Version doesn't conform
2. // to this format "HTTP/\d.\d", then the HTTP-Version fingerprint is "00"
3. {
4.     fp_n = '0';
5.     fp_v = '0';
6. }
7. else
8. {
9.     char fp_n = {};
10.
11.     if (http_name == "HTTP")
12.     {
13.         fp_n = '1';
14.     }
15.     else if (http_name == "http")
16.     {
17.         fp_n = '2';
18.     }
19.     else if (http_name == "Http")
20.     {
21.         fp_n = '3';
22.     }
23.     else
24.     {
25.         // other casing
26.         fp_n = '4';
27.     }
28.
29.     char fp_v = {};
30.
31.     if (major_ver == '0' && minor_ver == '8')        // = 0.8
32.     {
```

```
33.     fp_v = '1';
34. }
35. else if (major_ver == '0' && minor_ver == '9') // = 0.9
36. {
37.     fp_v = '2';
38. }
39. else if (major_ver == '1' && minor_ver == '0') // = 1.0
40. {
41.     fp_v = '3';
42. }
43. else if (major_ver == '1' && minor_ver == '1') // = 1.1
44. {
45.     fp_v = '4';
46. }
47. else if (major_ver >= '2' && minor_ver >= '0') // >= 2.0
48. {
49.     fp_v = '7';
50. }
51. else if (major_ver == '1' && minor_ver > '1') // = 1.( > 1)
52. {
53.     fp_v = '8';
54. }
55. else if (major_ver == '0' && minor_ver < '8') // = 0.( < 8)
56. {
57.     fp_v = '9';
58. }
59. else
60. {
61.     // it should never reach this case
62.     fp_v = 'f';
63. }
64. }
```

HTTP-Version fingerprint consists of two characters, the first encodes the HTTP string and the second encodes the HTTP version number.

For example, for the HTTP-Version “HTTP/1.1”, the fingerprint is “14”.

Status-Code Fingerprint (1-byte)

The algorithm for fingerprinting the Status-Code is as follows:

1. By the RFC standard, the Status-Code is a 3-digit code.
2. First, it is checked against a list of 102 known and “standardized” status codes, and if found in the list, the index to that value, plus 1, is taken as the fingerprint in hexadecimal. Otherwise, it is set to “ff”.
3. If the Status-Code is not in the Status-Line or doesn’t consist of 3 digits, the fingerprint is set to “00”.
4. The Status-Code 302 in this Status-Line "HTTP/1.1 302.1" is treated as valid, and only the value 302 is checked for. This is a design decision.

Status-Code List (note that the value (std::string) of the key is not accounted for. This is just for documentation purposes):

```
1. std::map<std::uint16_t, std::string> status_code
2. {
3.     // <https://http.dev/status>
4.     // Total: 102
5.
6.     // 09 - Informational responses
7.     {100, "Continue"},
8.     {101, "Switching protocols"},
9.     {102, "Processing"},           // WebDAV
10.    {103, "Early Hints"},
11.    {110, "Response is Stale"},     // Unofficial and Deprecated
12.    {111, "Revalidation Failed"},  // Unofficial and Deprecated
13.    {112, "Disconnected Operation"}, // Unofficial and Deprecated
14.    {113, "Heuristic Expiration"}, // Unofficial and Deprecated
15.    {199, "Miscellaneous Warning"}, // Unofficial and Deprecated
16.
17.    // 13 - Successful responses
18.    {200, "OK"},
19.    {201, "Created"},
20.    {202, "Accepted"},
21.    {203, "Non-Authoritative Information"},
22.    {204, "No Content"},
23.    {205, "Reset Content"},
24.    {206, "Partial Content"},
25.    {207, "Multi-Status"},         // WebDAV
26.    {208, "Already Reported"},     // WebDAV
27.    {214, "Transformation Applied"}, // Unofficial and Deprecated
28.    {218, "This is fine"},         // Unofficial
29.    {226, "IM Used"},             // HTTP Delta encoding
30.    {299, "Miscellaneous Persistent Warning"}, // Unofficial and Deprecated
31.
32.    // 09 - Redirection messages
33.    {300, "Multiple Choices"},
34.    {301, "Moved Permanently"},
35.    {302, "Found"},               // "Found" but used to be "Moved Temporarily"
36.    {303, "See Other"},
37.    {304, "Not Modified"},
38.    {305, "Use Proxy"},           // deprecated
39.
40.    // the 306 status code was defined in a previous version of this
41.    // specification, is no longer used, and the code is reserved
42.    {306, "Switch Proxy"},        // "Unused" but used to be "Switch Proxy"
43.    {307, "Temporary Redirect"},
44.    {308, "Permanent Redirect"},
45.
46.    // 44 - Client error responses
47.    {400, "Bad Request"},
48.    {401, "Unauthorized"},
49.    {402, "Payment Required"},    // Experimental
50.    {403, "Forbidden"},
51.    {404, "Not Found"},
52.    // Some servers return "Not Allowed", "Not Allowed." (wired.com)
53.    {405, "Method Not Allowed"},
54.    {406, "Not Acceptable"},
55.    {407, "Proxy Authentication Required"},
56.    {408, "Request Timeout"},
57.    {409, "Conflict"},
58.    {410, "Gone"},
59.    {411, "Length Required"},
60.    {412, "Precondition Failed"},
61.    {413, "Content Too Large"},   // OR "Payload Too Large"
62.    {414, "URI Too Long"},
63.    {415, "Unsupported Media Type"},
64.    {416, "Range Not Satisfiable"},
}
```



```

65.     {417, "Expectation Failed"},
66.     // <https://httpwg.org/specs/rfc9110.html#status.418>
67.     {418, "I'm a teapot"},           // "Unused" but used to be "I'm a teapot"
68.     {419, "Page Expired"},           // Unofficial
69.     {420, "Method Failure"},         // OR "Enhance your calm" Unofficial
70.     {421, "Misdirected Request"},
71.     {422, "Unprocessable Content"},   // OR "Unprocessable Entity" WebDAV
72.     {423, "Locked"},                 // WebDAV
73.     {424, "Failed Dependency"},       // WebDAV
74.     {425, "Too Early"},               // Experimental
75.     {426, "Upgrade Required"},
76.     {428, "Precondition Required"},
77.     {429, "Too Many Requests"},
78.     {431, "Request Header Fields Too Large"},
79.     {440, "Login Time-Out"},          // Unofficial
80.     {444, "No Response"},             // Unofficial
81.     {449, "Retry With"},              // Unofficial
82.     {450, "Blocked by Windows Parental Controls"}, // Unofficial
83.     {451, "Unavailable For Legal Reasons"},
84.     {460, "Client Closed Connection Prematurely"}, // Unofficial
85.     {463, "Too Many Forwarded IP Addresses"}, // Unofficial
86.     {464, "Incompatible Protocol"},   // Unofficial
87.     {494, "Request Header Too Large"}, // Unofficial
88.     {495, "SSL Certificate Error"},    // Unofficial
89.     {496, "SSL Certificate Required"},  // Unofficial
90.     {497, "HTTP Request Sent to HTTPS Port"}, // Unofficial
91.     {498, "Invalid Token"},            // Unofficial
92.     {499, "Token Required or Client Closed Request"}, // Unofficial
93.
94.     // 27 - Server error responses
95.     {500, "Internal Server Error"},
96.     {501, "Not Implemented"},
97.     {502, "Bad Gateway"},
98.     {503, "Service Unavailable"},
99.     {504, "Gateway Timeout"},
100.    {505, "HTTP Version Not Supported"},
101.    {506, "Variant Also Negotiates"},
102.    {507, "Insufficient Storage"},      // WebDAV
103.    {508, "Loop Detected"},             // WebDAV
104.    {509, "andwidth Limit Exceeded"},   // Unofficial
105.    {510, "Not Extended"},
106.    {511, "Network Authentication Required"},
107.    {520, "Connection Timed Out"},       // Unofficial
108.    {521, "Web Server Is Down"},         // Unofficial
109.    {522, "Origin Is Unreachable"},     // Unofficial
110.    {523, "Web Server Is Down"},        // Unofficial
111.    {524, "A Timeout Occurred"},        // Unofficial
112.    {525, "SSL Handshake Failed"},       // Unofficial
113.    {526, "Invalid SSL Certificate"},    // Unofficial
114.    {527, "Railgun Listener to Origin"}, // Unofficial
115.    {529, "The Service Is Overloaded"},  // Unofficial
116.    {530, "Site Frozen"},                // Unofficial
117.
118.    // <https://www.ietf.org/archive/id/draft-divilly-status-555-00.html>
119.    {555, "User Defined Resource Error"},
120.    {561, "Unauthorized"},               // Unofficial
121.    {598, "Network Read Timeout Error"}, // Unofficial
122.    {599, "Network Connect Timeout Error"}, // Unofficial
123.    {999, "Request Denied"}              // Unofficial
124. };

```

Therefore, the fingerprint for the status code 200 is "0a" (index of 9, + 1).

HTTP-Reason Fingerprint (2-byte)

The algorithm for fingerprinting the HTTP-Reason is described below. It's important to note that handling the HTTP-Reason requires a nuanced approach, incorporating specific design decisions that may intentionally deviate from strict RFC standards for practical reasons.

1. Before the HTTP-Reason itself can be obtained, the algorithm must first successfully match the HTTP-Version and Status-Code. Following this, it deliberately uses flexible parsing for the reason phrase. This flexibility is crucial because strict adherence to the standard status line format would cause subsequent probing checks to fail for many real-world responses that have minor discrepancies.

This is exercised via the following regex:

```
std::regex regexp("^HTTP/\\d\\.\\d \\d{3}([^\"]*)", std::regex_constants::icase);
```

For example, the reason phrase would be "potato":

```
HTTP/1.1 400Potato\ranother potato\r\n
```

1. The Reason-Phrase would be "potato", the last word.
 2. The `statusText()` method retrieves the reason phrase beginning from the first space after the status code. It continues searching, even if the status line ends with a newline, until it encounters a carriage return followed by a newline sequence. And, the same goes for the status code.
 3. Certain HTTP libraries might not be able to parse the HTTP-Reason if only CR or LF exists somewhere in the phrase before the marked ending of the Status-Line, that's CRLF.
 4. I managed to pull the entirety of the HTTP-Reason using the log API of Chilkat CkHTTP class. (Ref: **`response::status_line::get_resp_deep_log(CkHttp& http)`**)
2. If the HTTP-Reason is empty, the fingerprint is set to "0000".
 3. If the HTTP-Reason is not empty,
 1. If the reason phrase contains any character not in the std restricted ***achars***, then the algorithm checks for the following two conditions and **tags** the fingerprint to either of the unique values:
 1. "0001" if the Status-Code is followed by an SP, otherwise
 2. "0002"

```
1. // lambda function executed in-place
2. std::string achars = []()
3. {
4.     std::string achars = {};
5.     // obs-text [0x80-0xff]
6.     for (unsigned char i = 0x00; i <= 0x7f; ++i) { achars += (0x80 + i); }
7.     // SP / VCHAR [0x20-0x7e]
```

```
8.     for (unsigned char i = 0x00; i <= 0x5e; ++i) { achars += (0x20 + i); }
9.     // HTAB
10.    achars += 0x09;
11.    return achars;
12. }();
```

2. Otherwise, **FNV-1a** hash the HTTP-Reason, and take the upper two bytes of the hash.

Therefore, the fingerprint for the HTTP-Reason "OK" is "85e4", derived from FNV-1a_hash("OK") = 0x85e4b82f = 85e4.

While using only the upper two bytes from FNV-1a might seem to increase collision risk, this isn't a practical concern. The specific HTTP-Reason context limits the possibilities, and crucially, scans of the top 1 million Majestic websites found no collisions.

Moreover, the hashing algorithm **FNV-1a** was chosen because of its simplicity, speed, small hash value (4 bytes), and resistance to collisions compared to similar hashes.

Tagging the HTTP-Reason:

The probability of the designated tags ("0001", "0002") colliding with a valid hash is considered very low. This assessment is based on the inherent uniqueness of the tags, corroborated by empirical scans of the top 1 million Majestic websites, which yielded no corresponding hash matches.

Finally, the fingerprint of the Status-Line is the concatenation of the previous fingerprints:

(HTTP-Version | Status-Code | HTTP-Reason) = (14 | 0a | 85e4) = 140a85e4

P3/P4 Probes - Reason-Phrase Exception

P3 probes generate HTTP requests with a random resource string, and P4 probes with a random verb string. Specifically, upon receiving a P3 request, some servers will include the generated resource in the Reason-Phrase to signal its absence. Likewise, some servers responding to P4 requests will include the generated verb in the Reason-Phrase to indicate its prohibition. In these scenarios, the echoed resource string (for P3) or verb (for P4) is removed from the Reason-Phrase before applying the FNV-1a hash.

The table below provides examples of status-lines from servers exhibiting this behaviour in response to P3 requests (about 451 of the top million Majestic websites' servers show this behaviour):

Domain	HTTP Response - Status-Line
--------	-----------------------------

800pharm.com	HTTP/1.1 404 Page not found: / <u>MAAdqL9vaPTTAmbP</u>
accessmore.com	HTTP/1.1 404 / <u>hReEdqHe1qUTO9J5</u>
altruja.de	HTTP/1.1 404 Event not found with slug: <u>v3JfOERNPFmd9dPm</u>
billedbladet.dk	HTTP/1.1 404 Not Found "/ <u>O3N6TJCym8xbd3h6</u> "
bilpriser.dk	HTTP/1.1 400 Invalid path / <u>Z9pvfZYXzJBxDf3</u> was requested
dostoyanieplaneti.ru	HTTP/1.1 404 Component Not Found (<u>https://dostoyanieplaneti.ru/yA6Hf6Q9VL427nOO</u>)
eltonjohn.com	HTTP/1.1 404 The page you are looking for may not exist or may have moved: <u>ZZL4EeviqSCqiXIg</u>
pilkanozna.pl	HTTP/1.1 404 Nie znaleziono komponentu (<u>http://pilkanozna.pl/2mc5yTBgrefQjPxp</u>)
timeform.com	HTTP/1.1 404 Couldn't find a subheader with url segments <u>hMMsHWKC90K74VHE</u> and .

The underlined strings in the second column represent the pseudo-randomly generated resource strings. These are removed before the remaining portion of the Reason-Phrase is hashed. Observe that in some instances, the Reason-Phrase also contains the requested server's URL. Consequently, the resulting fingerprint becomes specific to that domain because the hash incorporates the target domain.

The table below provides examples of Status-Lines from servers exhibiting this behaviour in response to P4 requests (about 163 of the top million Majestic websites' servers show this behaviour):

Domain	HTTP Response - Status-Line
vetrf.ru	HTTP/1.1 501 Method <u>BRRKZNQVXV</u> is not defined in RFC 2068 and is not supported by the Servlet API
viafrance.com	HTTP/1.1 501 Unsupported method (' <u>VWYEQKTUPY</u> ')
viapresse.com	HTTP/1.1 405 HTTP method ' <u>ZWHAXAYFXN</u> ' is not allowed !
walkabout.com.au	HTTP/1.1 405 Method not allowed: <u>IDGXOBDKYA</u>
warzone.com	HTTP/1.1 501 Method <u>JTZTIUDBKI</u> is not defined in RFC 2068 and is not supported by the Servlet API
cibersur.com	HTTP/1.1 501 El Metodo <u>TTXYVEHBS</u> no esta definido en la especificacion RFC 2068 y no es soportado por la API Servlet

dancenter.de	HTTP/1.1 405 Method <u>ZQPEXTSQUAU</u> not implemented.
opony.com.pl	HTTP/1.1 405 Request method ' <u>LQTMQQKIRL</u> ' not supported
una.fr	HTTP/1.1 501 Le mode <u>VEHCPLTMUN</u> n'est pas dnée dans la RFC 2068 et n'est pas supporté par l'API Servlet

The underlined strings in the second column represent the pseudo-randomly generated verb strings. These are removed before the remaining portion of the Reason-Phrase is hashed.

Content-Length & Transfer-Encoding Fingerprint (C-L_T-E) (1-byte)

With probes P2-P5 and P8, the algorithm looks for the `Content-Length` header first. If it's there, it fingerprints that header. If not, it then looks for the `Transfer-Encoding` header and fingerprints that one if it exists.

The computation of this step is contingent on the validity of the Status-Line before proceeding, which is determined as follows:

```
1. std::smatch matches;
2. std::regex sl_rgx("^HTTP/\\d\\.\\d \\d{3}([\\^]*)", std::regex_constants::icase);
3.
4. if (!status_line.empty() && std::regex_search(status_line, matches, sl_rgx))
5. {
6.     std::string reason = matches[1];
7.
8.     // if the reason phrase contains any characters not specified in achars, then it does
9.     // not conform to the standard.
10.    if (reason.find_first_not_of(response::status_line::achars) != std::string::npos)
11.    {
12.        return false;
13.    }
14.    else
15.    {
16.        return true;
17.    }
18. }
19. if (!status_line.empty() && !std::regex_search(status_line, matches, sl_rgx))
20. {
21.     return false;
22. }
23. if (status_line.empty()) // simply, "valid" empty
24. {
25.     return true;
26. }
27.
```

1. If the validation against the Status-Line fails, then the **C-L_T-E** fingerprint is set to “99”.
2. If neither of the headers exists, then the **C-L_T-E** fingerprint is computed as follows:

The algorithm checks if the server response has any data in the body nonetheless, and if no data is present, the C-L_T-E fingerprint is set to "00", otherwise, the length of the data is encoded and the **C-L_T-E** fingerprint is the concatenation of "1" with `encode_length(body_data.getSize())`, where the function *encode_length* is defined as follows:

```
1. std::string response::content_length::encode_length(const std::uint32_t& length)
2. {
3.     if (length == 0)
4.     {
5.         return "0";
6.     }
7.     else if (length == 1)
8.     {
9.         return "1";
10.    }
11.    else if (length > 1)
12.    {
13.        return "2";
14.    }
15.    else
16.    {
17.        // should never reach here
18.        return "f";
19.    }
20. }
```

3. If the Content-Length header exists, then the **C-L_T-E** fingerprint is the concatenation of:

```
(encode_content_length_name(cl_name) | encode_length(resp-
>get_ContentLength()))
```

Where the function `encode_content_length_name()` is defined as follows:

```
1. std::string response::content_length::encode_content_length_name(const std::string& cl_name)
2. {
3.     if (cl_name == "Content-Length")
4.     {
5.         return "2";
6.     }
7.     else if (cl_name == "content-length")
8.     {
9.         return "3";
10.    }
11.    else if (cl_name == "Content-length")
12.    {
13.        return "4";
14.    }
15.    else if (cl_name == "content-Length")
16.    {
17.        return "5";
18.    }
19.    else if (helper::lowercase_str_out(cl_name) == "content-length")
20.    {
21.        return "6";
22.    }
23.    else
```

```
24.     {  
25.         return "f"; // you shouldn't get here!  
26.     }  
27. }
```

4. If the Transfer-Encoding header with the value “chunked” exists (anywhere in the value space), then the **C-L_T-E** fingerprint is the concatenation of:

```
encode_transfer_encoding_name(te_name) |  
encode_length(body_data.getSize())
```

Where the function `encode_transfer_encoding_name()` is defined as follows:

```
1. std::string response::content_length::encode_transfer_encoding_name(const std::string& te_name)  
2. {  
3.     if (te_name == "Transfer-Encoding")  
4.     {  
5.         return "7";  
6.     }  
7.     else if (te_name == "transfer-encoding")  
8.     {  
9.         return "8";  
10.    }  
11.    else if (te_name == "Transfer-encoding")  
12.    {  
13.        return "9";  
14.    }  
15.    else if (te_name == "transfer-Encoding")  
16.    {  
17.        return "a";  
18.    }  
19.    else if (helper::lowercase_str_out(te_name) == "transfer-encoding")  
20.    {  
21.        return "b";  
22.    }  
23.    else  
24.    {  
25.        // return 'e' instead of 'f' as in encode_content_length_name() to make it specific to  
TE  
26.        return "e"; // you shouldn't get here!  
27.    }  
28. }
```

As an example, for a Content-Length response like this:

```
HTTP/1.1 400 Bad Request  
Mime-Version: 1.0  
Content-Type: text/html  
Content-Length: 326  
Cache-Control: max-age=0  
Date: Mon, 07 Apr 2025 18:34:06 GMT  
Connection: keep-alive
```

The **C-L_T-E** fingerprint is “22” (the first “2” encodes the case sensitivity of the “Content-Length” header name, and the last “2” encodes the header length value (>1)).

As an example, for a Transfer-Encoding response like this:

```
HTTP/1.1 200 OK
Server: nginx
Date: Thu, 20 Mar 2025 12:25:20 GMT
Content-Type: text/html; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Strict-Transport-Security: max-age=31536000
Vary: Accept-Encoding
X-ac: 1.hhn_dfw MISS
Alt-Svc: h3=":443"; ma=86400

Hello World
```

The **C-L_T-E** fingerprint is “72” (the “7” encodes the case sensitivity of the “Transfer-Encoding” header name, and the “2” encodes the response body length (>1).

Strict-Transport-Security (HSTS) Header Fingerprint (1-byte)

For probe P1, in addition to the Status-Line fingerprint, the Strict-Transport-Security (HSTS) header is also fingerprinted, in great detail. The HSTS header tells the client/browser that the site should only be accessed using HTTPS. It takes multiple directives.

Just as in the case of the **C-L_T-E** case, the computation of this step is also contingent on the validity of the Status-Line before proceeding. If it is invalid, the **HSTS** fingerprint is set to “99”.

If the HSTS header doesn’t exist, the fingerprint is set to “00”.

The HSTS takes the following form, as an example:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

The HTTP Strict-Transport-Security (HSTS) standard specifies the attributes: `max-age` and `includeSubDomains`. The `preload` directive is non-standard². These attributes are separated by semicolons, and neither their sequence nor their capitalization affects how they are interpreted.

The **HSTS** fingerprint is a 1-byte (only the first 7 bits are taken) value that bit encodes several data formats as shown in the following table:

Bit Position to Set	Reason (HSTS Field Attributes)
---------------------	--------------------------------

² <https://hstspreload.org/>

[0]	exist(max-age)
[1]	if(max-age == 0) (regex("^max-age=0+\$"))
[2]	if(max-age).empty()
[3]	exist(includeSubDomains)
[4]	exist(preload)
[5]	if(more than three fields/attributes) - this will catch duplicates or any other additional attributes
[6]	exist(empty attribute). At least one empty attribute (ex., max-age=25;;)

All occurrences of the HSTS header are accounted for.

Consequently, the fingerprint for the **HSTS** header `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` is "19"; This hexadecimal value corresponds to the bitset `00011001`. If the bitset were all zeros (meaning an HSTS header is present but has no attribute values), the fingerprint would be set to "ff".

Content-Encoding (CE) Header Fingerprint (1-byte)

For probes P6 and P7, the Content-Encoding header is fingerprinted as per the Accept-Encoding header requests sent to the server. What compression algorithm(s) did the server select?

Just as in the case of the **C-L_T-E** case, the computation of this step is also contingent on the validity of the Status-Line before proceeding. If it is invalid, the **CE** fingerprint is set to "99".

If the **CE** header doesn't exist, the fingerprint is set to "00".

All occurrences of the **Content-Encoding** header are accounted for.

For example,

```
Content-Encoding: gzip, br, compress, x-gzip
Content-Encoding: deflate
```

The list of compression algorithms that are sent with the Accept-Encoding header in the P6 probe is also used here as a reference point for determining whether the server picks any of them:

List of Accept-Encoding compression algorithms (referred to as A-E_List):

```
1. std::array<std::string, 13> accept_encoding_v =
2. {
3.     "aes128gcm", "br", ae_grease_1, "compress", "deflate",
```

```
4.     "exi", ae_grease_2, "gzip", "pack200-gzip", "x-compress",
5.     "x-gzip", "zstd", "identity"
6. };
```

The **CE** fingerprint is a 1-byte value that bit encodes several data formats as shown in the following table:

Bit Position to Set	Description
[0-3]	<pre>if (!exist(A-E_List)) { [0-3] = 0x0f (15) } else { [0-3] = Index(A-E_List) + 1 }</pre>
[4]	If value contains an empty sequence (ex., Content-Encoding: ,)
[5-7]	Number of additional compression algorithms (if more than one value is returned). The max number these bits could store is 7. <pre>if ([5-7] > 3) { [5-7] = 7 }</pre> The if statement is needed in case [0-3]=0x09 and [4] is set. A collision with the fingerprint value of "99" is possible in case the [5-7] = 4.

Consequently, the **CE** header `Content-Encoding: deflate` fingerprint is "05"; This hexadecimal value corresponds to the bitset `00000101`.

Allow, Access-Control-Allow-Methods (ACAM) and Public Headers Fingerprint (3-byte)

In probe P7, the server response is examined for the presence of `Allow`, `Access-Control-Allow-Methods` (ACAM), and `Public` HTTP headers (collectively, we refer to all the headers as **AACAMP**). All identified allowed HTTP verbs across these headers are collected, bit-encoded, and a normalized list is then subjected to FNV-1a hashing. This generates a final 3-byte (6 hexadecimal characters) fingerprint. The detailed methodology is presented below. This fingerprint is referred to as **AAP**.

Just as in the case of the **C-L_T-E** case, the computation of this step is also contingent on the validity of the Status-Line before proceeding. If it is invalid, the **AAP** fingerprint is set to "999999".

The **AAP** fingerprint consists of:

AAP_FP = <1-byte_bitset_encoded_length><2-bytes_fnv1a_hash_headers>

Let's take this response as an example:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Headers: accept, genx-auth, genx-ua, genx-ip
Access-Control-Max-Age: 86400
Allow: OPTIONS,GET,HEAD,Trace
Allow: POST
Date: Thu, 20 Mar 2025 12:25:40 GMT
Connection: keep-alive
Public: POST, TRACE
Keep-Alive: timeout=5
Transfer-Encoding: chunked
```

Typically, the allowed HTTP verbs within each of these headers (`Allow`, `Access-Control-Allow-Methods`, `Public`) are separated by commas. In this particular response, all three headers are present, showcasing duplicate verbs, variations in case sensitivity, and instances where verbs are split across multiple `Allow` headers. Importantly, the algorithm considers all occurrences of each header.

The <2-bytes_fnv1a_hash_headers> part is calculated as follows:

The process begins by iterating through all occurrences of the `Allow`, `Access-Control-Allow-Methods`, and `Public` headers. Within each header, the comma-separated verbs are extracted and added to this header type vector. This vector of verbs is then sorted alphabetically, and all whitespace is eliminated. After this normalization per header type, the sorted verb lists are concatenated in the sequence: `Allow`, followed by `ACAM`, and lastly `Public`. The final step involves calculating the **FNv-1a** hash of this concatenated string and taking its upper two bytes.

For the example shown above, this would translate to:

1. Get all verbs per header type and sort them into separate vectors:
 1. **Allow**: { "GET", "HEAD", "OPTIONS", "POST", "Trace" }
 2. **ACAM**: { "GET", "OPTIONS", "POST" }
 3. **Public**: { "POST", "TRACE" }
2. Concatenate all the values in the vectors into one string as per the order listed above:
"GETHEADOPTIONSPOSTTraceGETOPTIONSPOSTPOSTTRACE"
3. **FNv-1a_Hash**("GETHEADOPTIONSPOSTTraceGETOPTIONSPOSTPOSTTRACE") and take the upper two bytes. This yields the 32-bit hash 0xd02f8e3f = "d02f". If no verbs exist, then the value is "0000".

The <1-byte_bitset_encoded_length> part is calculated as follows:

Bit Position to Set	Description
[0]	header_exist("Allow")
[1]	header_exist("Access-Control-Allow-Methods")
[2]	header_exist("Public")
[3-7]	The last 5 bits are for storing the total number of verbs for all the headers, for a total value of 31. <pre>// between 0 and 30 if (aacamp_total >= 0 && aacamp_total < 31) { aacamp_total_nb = aacamp_total; } else if (aacamp_total > 30) { aacamp_total_nb = 31; }</pre>

For the above server response, the <1-byte_bitset_encoded_length> part translates to: **01001111**, which is 0x4f in hexadecimal. Hence, the fingerprint for <1-byte_bitset_encoded_length> is "4f".

Therefore, the **AAP_FP** is "4fd02f".

Connection Header Fingerprint (2-byte)

For probes P2-P5 and P7, a 2-byte (4 hexadecimal characters) bit-encoded fingerprint tracks the `Connection` header, specifically noting if its value is "Keep-Alive" or "Close". The server response for each of these probes is checked for the presence of the `Connection` header and these values. Notably, even if a server violates HTTP protocol by including the `Connection` header multiple times with differing values, the fingerprint will register all such instances. All checks are done case-insensitively.

The connection fingerprint is denoted by:

Cnx_FP: <1-byte_bitset_encoded_keep_alive><1-byte_bitset_encoded_close>

Because the Connection fingerprint (**Cnx_FP**) tracks data for only 5 probes, each of its two bytes uses its first 5 bits to represent the status for each of those probes.

For example, if server A responds with the headers:

Date: Wed, 19 Mar 2025 16:09:49 GMT

Content-Type: text/plain

Content-Length: 0

Connection: keep-alive, close

In this case, the Connection header value doesn't honour the specification, but the **Cnx_FP** works by accounting for both values.

Another example, if server B responds with the headers:

Date: Wed, 19 Mar 2025 16:09:49 GMT

Content-Type: text/plain

Content-Length: 0

Connection: keep-alive

Connection: close

The Connection header repeats with different values in this case, and the **Cnx_FP** saves both occurrences.

For computing <1-byte_bitset_encoded_keep_alive>:

Probe/Value	P2[0]	P3[1]	P4[2]	P5[3]	P7[4]
Keep-Alive	Set/Unset	Set/Unset	Set/Unset	Set/Unset	Set/Unset

For computing <1-byte_bitset_encoded_close>:

Probe/Value	P2[0]	P3[1]	P4[2]	P5[3]	P7[4]
Close	Set/Unset	Set/Unset	Set/Unset	Set/Unset	Set/Unset

Verbosus Fingerprint (38-byte)

As noted in the Abstract, the verbosus fingerprint is a 38-byte (76-character) string structured as follows:

```
verbosus: 01[0-9A-Fa-f]{74}
```

The initial byte, "01", indicates the hash version, while the subsequent 37 bytes contain the combined fingerprints from all probes (P1-P8) and the Connection fingerprint (Cnx_FP).

It's important to note that version number "02" cannot be used for any new verbosus fingerprints. This is because "02" is specifically reserved for the Pacto fingerprint.

The complete verbosus fingerprint is constructed by concatenating the individual fingerprints of each probe in the following sequence:

Verbosus_fp: P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | Cnx_FP

Probes	Len	Fingerprint Headers* (concatenated)
P1 - GET Normal - Valid Request	5	Status-Line (4) Strict-Transport-Security (1)
P2 - GET Invalid HTTP Version Request	5	Status-Line (4) Content-Length (1)
P3 - GET Random Resource Request	5	Status-Line (4) Content-Length (1)
P4 - Random Verb Request	5	Status-Line (4) Content-Length (1)
P5 - get Lowercase Verb	5	Status-Line (4) Content-Length (1)
P6 - GET Request - Accept-Encoding (Full)	1	Content-Encoding
P7 - GET Request - Accept-Encoding (Less)	1	Content-Encoding
P8 - OPTIONS Request	8	Status-Line (4) Content-Length (1) (Allow Access-Control-Allow-Methods Public) (3)
CONX _(P2,P3,P4,P5,P7)	2	Connection: Keep-Alive (1) Connection: Close (1)
P1 ... P8 CONX	37	

For example, using the HTTP-Basma tool, we can get the fingerprint for the domain example.com with the following command:

```
HTTPBasma.exe --domain example.com --port 80
```

Verbosus_fp:

```
01140a85e4001420958a22142494d6221320958a221420958a220800144d9e7f220000001b04
```

Which is dissected as follows:

```
P1: 140a85e400
P2: 1420958a22
P3: 142494d622
P4: 1320958a22
P5: 1420958a22
```

P6: 08
P7: 00
P8: 144d9e7f22000000
Cn: 1b04

The tool can generate CSV and JSON files containing specific information about each of the probes' responses, with complete dissection of the **verbosus** fingerprint.

The CSV file contains several columns, including the expected data alongside specific information for each probe. This probe-specific data includes the fingerprint (p1_fp, p2_fp, etc.), the response status line (p1_sl, p2_sl, etc.), and the raw response headers (p1_rh, p2_rh, etc.). Additionally, an experimental fingerprint of the response headers is present for each probe under columns named p1_rh_fp, p2_rh_fp, etc. (more details on this are available in the Tool section).

In both the CSV and JSON output formats, probes P6, P7, and P8 are represented by the column/field names p6f_fp, p6l_fp, and p7a_fp, respectively.

Pacto Fingerprint

The **pacto** fingerprint is 32 bytes long with the format:

pacto_fp: <1-byte-pacto-identifier><first-15-bytes-sha256-hash-of-verbosus_fp>

For the domain example.com, the Pacto fingerprint (pacto_fp) is
025d3bb410ad49ad67327bf930ef5a8a.

The initial "02" indicates it's a Pacto fingerprint. The subsequent 15 bytes (5d3bb410ad49ad67327bf930ef5a8a) are derived by first calculating the SHA-256 hash of the verbosus fingerprint (excluding its version byte) and then taking the first 15 bytes of that hash.

For instance, if the verbosus fingerprint (without the version byte) is 140a85e4001420958a22142494d6221320958a221420958a220800144d9e7f220000001b04, its SHA-256 hash when truncated to the first 15 bytes results in 5d3bb410ad49ad67327bf930ef5a8a, which is then prepended with "02" to form the final Pacto fingerprint.

If the **verbosus_fp** is all zeros, then the **pacto_fp** is set to zeros.

Tool - HTTP-Basma

HTTP-Basma is a C++ tool I developed to showcase the practicality and viability of this algorithm. It leverages Chilkat's³ library for all HTTP socket interactions and utilizes other supporting classes within the library. Additionally, the tool includes a demangler feature that can dissect and reverse the verbusus fuzzy-hash, outputting a comprehensive JSON object.

Be aware that some output of the tool might use slightly different probe numbers, but the underlying order remains consistent: P1->P1, P2->P2, P3->P3, P4->P4, P->P5, P6->P6F, P7->P6L, P8->P7a.

When requesting a given domain/IP, the response could be saved to a CSV or JSON file with a plethora of information about the server response headers and each probe's unique fingerprint.

The tool supports HTTP redirect (enabled by default).

- An experimental fingerprinting algorithm for the response headers is included in the tool for comparison purposes. It works as follows:
 1. Get all the response headers except for "Connection: Keep-Alive", "Connection: Close" and any header that starts with "X-" (custom header). All checks are performed case-insensitively
 2. Save all unique header names into a vector
 3. Sort the vector alphabetically
 4. Concatenate all the sorted headers into one string
 5. FNV-1a hash the final string and take the full 32-bit hash as the fingerprint
 6. If no headers are present in the server response, the fingerprint is set to zero

For demonstration, take the following server response headers:

```
Date: Wed, 19 Mar 2025 15:02:53 GMT
Content-Type: text/html; charset=UTF-8
Cache-Control: max-age=1
Connection: keep-alive
X-Powered-By: ASP.net
Content-Length: 0
Sever: Microsoft-IIS/10.0
```

The final string before hashing is: "Cache-ControlContent-LengthContent-TypeDateSever".

The FNV-1a hash of the string "Cache-ControlContent-LengthContent-TypeDateSever" is "df0062c6".

The Demangler

³ <https://www.chilkatsoft.com/>

The tool's demangler function `"-i/--demangle_json"` takes a verbose fingerprint and reconstructs the attributes of each probe, outputting a comprehensive JSON object. Notably, when attempting to reverse the FNV-1a hashes, the demangler utilizes two local databases: `options.csv` for allowed HTTP methods and `status_line_db.csv` for status-line reason phrases. If either of these database files is missing, the corresponding hash reversal feature is automatically disabled. These databases were compiled from a scan of the top 1 million Majestic websites.

The demangling of the verbose's fingerprint, for the domain `example.com`:

`"01140a85e4001420958a22142494d6221320958a221420958a220800144d9e7f220000001b04"`

Within the JSON file, the `sl_reversed_db` object for the status-line contains the following data:

```
1.     "sl_reversed_db": {
2.       "http_version": "HTTP/1.1",
3.       "status_code": [200,404,403,500,204,999,888,603],
4.       "http_reason": "OK"
5.     }
```

Observe that the `"status_code"` array contains multiple HTTP status codes. This occurs because different servers might use the same reason phrase for different status codes, resulting in identical FNV-1a hashes.

Consider another example: for the domain `"0009.in"`, the verbose fingerprint is `"01140a85e400145183b23200000000001420958a321420958a320202140a85e430216e12100d"`, and its corresponding `"allow"` options data in the JSON output is as follows:

```
1.  "allow_hdr": {
2.    "fp": "216e12",
3.    "hdr": {
4.      "fp": "21",
5.      "methods": {
6.        "total": 4,
7.        "total_cmt": "total number of allowed methods across all allow headers' types",
8.        "hash": "6e12"
9.      },
10.     "hdrs": [
11.       "allow"
12.     ]
13.   },
14.   "hdrs_value": {
15.     "allow": [
16.       "OPTIONS,HEAD,GET,POST"
17.     ]
18.   }
19. }
```

As illustrated in the `"hdrs_value"` JSON object above, the `"allow"` array contains the HTTP methods `"OPTIONS,HEAD,GET,POST"`. These methods correspond to the FNV-1a hash `"6e12"`, which was retrieved from the `options.csv` database file.

The Comparator Option

The comparator option “-C/--compare” compares two verbose fingerprints and prints out the differences across the major components for each of the probes.

For example, comparing the following two fingerprints

```
HTTPBasma.exe --compare
01140a85e40014514bd522142494d622144d9e7f221420958a220800140a85e42
02931de1609,01140a85e40014514bd522142494d622144d9e7f221420958a220
800140a85e42029e4a01609
```

Results in this output:

```
< FPrnt-1 Vs. FPrnt-2 >

[ P1 ]
[ P2 ]
[ P3 ]
[ P4 ]
[ P5 ]
[ P6F ]
[ P6L ]
[ P7a ]
{Allow Header(s) Hash}
allow header: 31de != e4a0
```

The output reveals a difference specifically in the hash component of the p7a probe's fingerprint.

Scanning a List of Domains

For batch scanning of domains, IPs, or URLs, provide the list using “-f/--file” and choose either “-c/--csv” or “-j/--json” for the output format (one of them is required). The tool runs sequentially by default; to perform the scan in parallel, use the “-P/--parallel” option.

Each domain has to be on a new line. Moreover, a line that starts with ‘;’, ‘//’ or ‘#’ is skipped.

If you specify a port using “-p/--port” or enable SSL with “-s/--ssl”, all domains in the scanned file will use that same port and SSL setting.

Each domain's port number and SSL status can be determined from its URL. An “https” prefix sets the SSL flag and the port number to 443. While the default port is 80 when unspecified and the prefix is not “https”, a colon (':') allows for explicit port definition. For example, https://www.nohereorthere.com:8083 shows both SSL active and port 8083 used. The same interpretation applies to the “-d/--domain” option.

Experimentation

For experimentation, I scanned the top 1 million Majestic⁴ websites and other malware C&C servers and frameworks. The actual number is 986,910 websites.

Note that the experimentation was carried out from February 2024 till late March 2024, so, some servers will have different fingerprints today.

Observations:

- Amazon.com and potentially other servers seem to exhibit an interesting behaviour when sending the same request to the server.
 - The observed changes in the number of response headers are likely due to the server being situated behind infrastructure components such as CDNs or load balancers. These intermediaries introduce request-specific headers that can vary. For instance, a cache miss on the Cloudflare CDN results in an "X-Cache: Miss from cloudfront" header. This behaviour differs from standalone servers, where the set of response headers generally remains consistent across requests.
 - I've observed that the case sensitivity of several header names can change. This includes, but isn't limited to, connection, server, content-type, and date, which are sometimes returned with all letters in lowercase.
- For whatever reason, the presence of the Connection header is not consistent when querying some servers' multiple times.
- According to the standard, when a server returns a 405 "Method Not Allowed" status for a resource, it should include an Allow header listing the supported methods. However, this isn't always implemented, with thousands of servers failing to include the Allow header in such responses.

C&C RAT Frameworks Fingerprints

CobaltStrike ^A
v: 011420958a0014514bd5221420958a221420958a221420958a2200001420958a220000000001f p: 02464ae8b7d86f82c9918e2c2b9d6b91 • Note: FP rate (72/986,910)
QakBot ^B

⁴ <https://majestic.com/reports/majestic-million>

v: 011423a9a80014514bd5221423a9a8221423a9a8221420958a2208001423a9a822000 000001f p: 02ebabc0dbf3212783eb5109e94518ec	5846
---	------

Some Interesting Content-Encoding Values

Examination of `Content-Encoding` header values across the top 1 million Majestic websites have uncovered instances that deviate from the defined standard, highlighting a lack of uniform adherence among servers.

`Content-Encoding: UTF-8`
Example: `abcmouse.com`
`Content-Encoding: utf-8`
Example: `cta.int`
`Content-Encoding:`
Example: `feedmelinks.com, steem.com, steem.io`
`Content-Encoding: identity, compress`
Example: `gminagostynin.pl`
`Content-Encoding: br, gzip`
Example: `madamemadeline.com`
`Content-Encoding: gzip, gzip`
Example: `rusprofile.ru`

`Content-Encoding: ⚡identity`

Example: `shoowbiz.ru`
`content-encoding: compress, gzip`
Example: `thptduongdong.com`
`content-encoding: AnyTrash`
Example: `urlgalleries.net`

Some Interesting Options Allow Header Values

Probe P8 yielded some unusual server responses to the `OPTIONS` request. The table below highlights examples of `Allow` header values that either don't follow the standard comma-separated syntax or are infrequently seen. This suggests that not all servers adhere to the `Allow` header specification.

Of particular interest is the servers responding with the `Allow` header set to an **empty** value, which signifies that the resource currently accepts no HTTP request methods. This situation might be temporary for a specific resource.

Allow Header
*
GET;HEAD;OPTIONS

GET, GET
OPTIONS, GET, HEAD, POST
HEAD, HEAD, GET, HEAD, POST, OPTIONS, TRACE
CLI, GET, POST
HEAD; HEAD; HEAD; GET
GET POST HEAD
ACL, BIND, CHECKOUT, CONNECT, COPY, DELETE, GET, HEAD, LINK, LOCK, M- SEARCH, MERGE, MKACTIVITY, MKCALENDAR, MKCOL, MOVE, NOTIFY, PATCH, POST, PROPFIND, PRO PPATCH, PURGE, PUT, REBIND, REPORT, SEARCH, SUBSCRIBE, TRACE, UNBIND, UNLINK, UNLOCK, U NSUBSCRIBE
CHECKOUT, CONNECT, COPY, DELETE, GET, HEAD, LOCK, M- SEARCH, MERGE, MKACTIVITY, MKCOL, MOVE, NOTIFY, PATCH, POST, PROPFIND, PROPPATCH, PURG E, PUT, REPORT, SEARCH, SUBSCRIBE, TRACE, UNLOCK, UNSUBSCRIBE
OPTIONS, GET, HEAD, TRACE, PROPFIND, PROPPATCH, MKCOL, COPY, PUT, DELETE, MOVE, LOCK, UNLOCK, BIND, REBIND, UNBIND, VERSION-CONTROL
, HEAD, POST, GET, HEAD, OPTIONS
*, *
POST,, HEAD,mÑ%fU, GET, HEAD, OPTIONS,, HEAD,, HEAD
GET, HEAD; GET, POST, HEAD, OPTIONS, PUT, DELETE
GET, GET, GET
get, post, head
get, head, delete, options
get
get, post
[DELETE, POST, GET, OPTIONS, PUT, PATCH]
REDIRECT GET HEAD POST PUT DELETE OPTIONS SEND FORWARD FAIL
<empty_no_value>
GET POST HEAD
GET HEAD OPTIONS
Access-Control-Allow-Methods Header
"PUT, GET, POST, DELETE, OPTIONS"
true
https://eu-app.contentstack.com
GET, POST, PATCH, PUT, DELETE, OPTIONS; GET, POST, PUT, DELETE, OPTIONS
GET, POST, PATCH, PUT, DELETE, OPTIONS; GET, POST, PUT, DELETE, OPTIONS

GET, HEAD, POST, PUT, DELETE, TRACE, OPTIONS, SCRIPT	
GET;GET, POST, PUT, DELETE, PATCH, PURGE, HEAD, OPTIONS, FETCH;GET, POST, PUT, DELETE, PATCH, PURGE, HEAD, OPTIONS, FETCH	
Content-Type	
*;GET, POST, PUT, DELETE, OPTIONS	
*;GET	
;	
'HEAD, GET, POST, PUT, PATCH, DELETE'	
'HEAD, GET, POST, PUT, PATCH, DELETE'	
*	
<empty_no_value>	
Public Header	
put, trace, options, delete, post, get, head	
OPTIONS, TRACE, GET, HEAD, POST X-Frame-Options: SAMEORIGIN	
OPTIONS, TRACE, GET, HEAD, POST;GET,POST,HEAD	

Status-Line Reason Phrases

The following is a list of some of the distinct status-line reason phrases we observed. In total, approximately 680 unique phrases were returned by the servers we analyzed.

A sample of status-line reason phrases:

```

unknown status
unknown method
unknown
undefined
page non trouvée
page non trouv
or worse
okay
ok
object_not_exists
nicht gefunden
method not implemented
Your user agent is banned. Please try with another web browser.
You are banned from this site. Please contact via a different client
configuration if you believe that this is a mistake.
You are banned from this site.
UNSUPPORTED MEDIA TYPE
Task [] not found
Really in Trouble
Oops! We can't find that page.
```

Ooooooops
Notfound
NotFound
Not-Found
Not in ip whitelisting
Not implemented
Nie znaleziono komponentu
Nicht gefunden
Naughty, not nice!
Naughty, not nice!
NOT_FOUND
NOTOK
NOT SUPPORTED METHOD
NOT IMPLEMENTED
NOT FOUND
NO MATCHING ROUTE for parameters {}
NO CONTENT
NAI
Internal Server Error
Internal Error
Interdit
Insufficient Storage
INTERNAL SERVER ERROR
I'm sorry, you has been banned
I'm a teapot.
I'm a teapot
I'M A TEAPOT
HttpException
Gone Walkabout
Gone
Get out of here!
Gebied niet gevonden
Gateway error
Gateway Timeout
Gateway Time-out
GONE
GATEWAY_TIMEOUT
Friendly Not Found

Conclusion

HTTP fingerprinting a server to generate a specific impression for identification is an ad-hoc approach of trial and error. However, creating a truly effective fingerprint, one that's reversible, modular, shareable, compact, and optimized, requires sophisticated techniques like bit-encoding, hashing, and data transformation. The HTTP-Basma verbosus fingerprint format is the first algorithm to successfully achieve these objectives.

This paper showcased HTTP-Basma's practical application and feasibility by successfully grouping fingerprints into clusters based on server configurations and behaviors. This was validated against both the top one million Majestic websites and malicious C&C servers.

Nevertheless, HTTP-Basma does have limitations, including its vulnerability to false positives and false negatives. A minor alteration, such as replacing a status-line reason-phrase, could lead to a different fingerprint for the identical server. And, depending on how you interpret such evasion technique, the identity of the server therefore has changed, which necessitates a new fingerprint. Moreover, inconsistencies in the server replying with the Connection header at different execution times, leads to different fingerprints.

Bibliography

- Kotarak, A. (2003, May 23). *WebServerFP-Source.zip*. Retrieved May 14, 2026, from PACKET STORM: <https://packetstorm.news/files/id/31157/>
- Lee, D. W. (2001). *HMAP: A Technique and Tool For Remote Identification of HTTP Servers*. University of California, Computer Science. Davis: Computer Security Lab. Retrieved May 14, 2026, from <http://seclab.cs.ucdavis.edu/papers/hmap-thesis.pdf>
- Ruef, M. (2007, December 06). *httprecon project - advanced web server fingerprinting*. Retrieved May 14, 2026, from computec.ch: <https://www.computec.ch/projekte/httprecon/>
- Shah, S. (2004, May 19). *An Introduction to HTTP fingerprinting*. Retrieved May 14, 2026, from Net Square: https://net-square.com/httpprint_paper.html